Studio Nx

AI 구현 설명 32 — P1-02 프론트 공통 JS

2026-10-07

AI 구현 설명 32 — P1-02 프론트 공통 JS (API 호출 · 로그인 확인)

항목	내용
작성	Claude (AI) · 2026-10-06
작업	P1-02 프론트 공통 JS — PRD 01 FR-104 · FR-105
브랜치	`feature/phase1/p1-02-frontend-api-core`
바꾼 파일	`studionx/frontend/static/js/core/api.js` · `studionx/frontend/static/js/core/auth.js`
진행 방식	**함수 하나씩** 구현 → 사용자 검토 → 다음 함수 (2026-10-06 규칙 변경)
함수별 설명	`docs/ai-notes/javascript/P1-02 - 프론트 공통 JS/01 ~ 08` (md · PDF), 합본 `javascript/P1-02 frontend api core.pdf`
이전 문서	31 sequencer buttons.md

1. 한 일

모든 페이지가 함께 쓰는 두 모듈을 구현했다.

모듈	역할
`core/api.js`	서버 API 호출 — 토큰을 붙이고, 만료되면 재발급 후 다시 보내고, 실패를 `ApiError` 하나로 통일
`core/auth.js`	로그인 확인 · 로그인 화면으로 보내기 · 돌아갈 주소 처리 · 로그아웃

2. 함수 8개

#	파일	함수	하는 일	설명
01	`api.js`	`refreshAccessToken()`	리프레시 쿠키로 Access Token 재발급 (동시에 여러 번 불려도 요청 1번)	01
02	`api.js`	`apiFetch(path, options)`	모든 API 호출 — 토큰 · 401 재시도 · 오류 형식	02
03	`auth.js`	`safeNextPath(next)`	돌아갈 주소가 우리 사이트 경로인지 검사 (열린 리다이렉트 방지)	03
04	`auth.js`	`loginUrl(next)`	`home.html#login?next=…` 주소 만들기	04
05	`auth.js`	`parseLoginHash(hash)`	`#login?next=…` 읽기 → 모달을 열지 · 돌아갈 곳	05
06	`auth.js`	`isLoggedIn()`	로그인 상태 확인 (토큰이 없으면 재발급 시도)	06
07	`auth.js`	`requireLogin()`	로그인 전용 페이지에서 로그인 안 됐으면 로그인 화면으로	07
08	`auth.js`	`logout()`	서버 쿠키 삭제 · 메모리 토큰 삭제 · 홈으로	08

3. 함수들이 함께 움직이는 모습

로그인 안 된 사람이 내 프로젝트(`/myProject.html#new`)를 열 때

순서	함수	결과
①	`requireLogin()` → `isLoggedIn()` → `refreshAccessToken()`	쿠키 없음 → `false`
②	`loginUrl('/myProject.html#new')` (안에서 `safeNextPath`)	`home.html#login?next=%2FmyProject.html%23new`
③	`location.replace(…)`	홈으로 이동, 방문 기록에서 내 프로젝트를 지움
④	`parseLoginHash()` (P1-07 `home.js`)	`{ open: true, next: '/myProject.html#new' }` → 로그인 모달 열림
⑤	로그인 성공 (P1-07)	`/myProject.html#new` 로 돌아옴

로그인한 사람이 페이지를 옮겼을 때 (메모리의 토큰이 사라진 상태)

순서	함수	결과
①	`api.get('/me/dashboard')` → `apiFetch`	토큰 없음 → 먼저 `refreshAccessToken()`
②	`POST /auth/refresh` (리프레시 쿠키가 자동으로 실림)	새 Access Token
③	`GET /me/dashboard` + `Authorization: Bearer …`	결과 JSON

30분이 지나 토큰이 만료되었을 때: `apiFetch` 가 401 `UNAUTHORIZED` 를 받으면 재발급 → 같은 요청을 **한 번만** 다시 보낸다.

4. 토큰 정리

토큰	어디에	수명	쓰임	지우는 곳
Access Token	메모리 (`api.js` 변수)	30분	모든 API 요청의 `Authorization` 헤더	`logout()` · 재발급 실패
리프레시 토큰	HttpOnly 쿠키 `snx_refresh` (`path=/api/v1/auth`)	자동 로그인 3일 · 아니면 브라우저를 닫을 때까지	Access Token 재발급	서버 (`POST /auth/logout`)

- Access Token 을 `localStorage` 에 두지 않는 이유: 스크립트 공격(XSS)이 읽어 갈 수 있다.
- 리프레시 쿠키를 `HttpOnly` 로 두는 이유: JS 에서 아예 읽을 수 없다.
- 주소의 `next` 에는 경로만 들어간다. 토큰 · 비밀번호는 주소에 들어가지 않는다 (04 보안).

5. 템플릿과 달라진 점

함수	템플릿	구현	이유
`safeNextPath`	`(next)`	`(next, origin = location.origin)`	시험에서 사이트 주소를 넣으려고. 평소에는 생략
`loginUrl`	경로를 그대로 붙임	`safeNextPath` 로 검사한 뒤 붙임	잘못된 값이 들어와도 위험한 주소를 만들지 않음
`requireLogin`	경로 + 쿼리	경로 + 쿼리 + **해시**	`#new`(새 프로젝트 모달) 같은 상태도 로그인 후 그대로
`logout`	`location.href`	`location.replace` · `auth: false`	뒤로 가기로 로그인 전용 화면이 다시 보이지 않게 · 로그아웃 중 재발급 방지
`apiFetch`	—	도우미 `toApiError()` 추가	401 처리와 일반 실패가 같은 변환을 씀

6. 시험

백엔드(`/auth/refresh` · `/auth/logout` 등)는 P1-03 · 04 · 05 에서 구현하므로, `fetch` · `location` 을 가짜로 바꿔 Node.js 에서 시험했다.

함수	시험 수	결과
`refreshAccessToken`	10	모두 통과
`apiFetch`	16	모두 통과
`safeNextPath`	14	모두 통과
`loginUrl`	8	모두 통과
`parseLoginHash`	12	모두 통과
`isLoggedIn`	5	모두 통과
`requireLogin`	7	모두 통과
`logout`	6	모두 통과

실제 서버와 주고받는 시험은 P1-05(로그인 · 재발급 · 로그아웃 API) 이후 P1-18 에서 한다.

7. 다음 작업이 쓰는 곳

작업	쓰는 함수
P1-07 가입 · 로그인 JS	`api.post` · `setAccessToken` · `parseLoginHash` · `safeNextPath` · `isLoggedIn`(헤더 버튼)
P1-10 · 14 · 17 페이지 JS	`requireLogin` · `api.get/post/patch/delete` · `logout`
시퀀서 `sequencer.js` `init()`	`requireLogin` · `api.get('/projects/{id}/full')`